

第1天 Java 基础语法

今日内容介绍

- ◆ Java 开发环境搭建
- ◆ HelloWorld 案例
- ◆ 注释、关键字、标识符
- ◆ 数据（数据类型、常量）

第1章 Java 开发环境搭建

1.1 Java 概述

众所周知 Java 是一门编程语言，编程语言就是用来编写软件的。那么使用 Java 到底能用来编写什么软件呢？你所熟知的很多软件都可以用 Java 来编写，例如：QQ、迅雷、淘宝、京东等等，那么 Java 比较擅长什么呢？也就是说我们将来从事 Java 开发工程师工作到底要编写什么类型的软件呢？

答案是：

1. 互联网：电商、P2P 等等；
2. 企业级应用：ERP、CRM、BOS、OA 等等。

互联网类软件大家都比较熟悉，因为大家生活都可以接触得到。而企业级应用都是为大型企业量身定做的软件，用于工作中，所以大家可能不太了解。现在大家只需对 Java 开发工程师工作有个大致的了解即可。

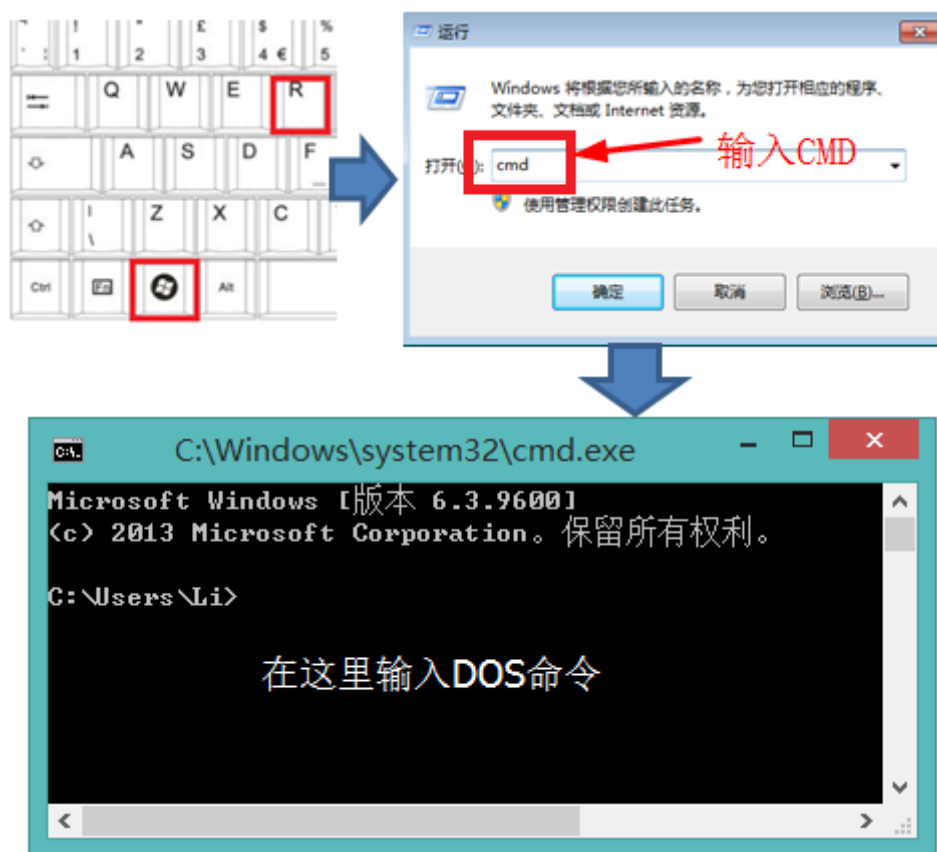
在基础班我们会学习 Java 的基础知识，即 JavaSE（标准版）部分。学习完 JavaSE 并不能开发大型项目。

在就业班我们会学习 Java 的高级知识以及整个 Java 生态圈，即 JavaEE（企业版）部分。学习完 JavaEE 部分就可以开发各种大型项目了。

1.2 常用 DOS 命令

编写源文件后就需要开始编译操作了，但在 Windows 操作系统中编译、运行都需要了解常用 DOS 命令，所以下面我们来学习一下常用 DOS 命令（DOS 命令是 Windows 操作系统包含的知识）。

DOS 命令是在 DOS 控制台中使用的命令，弹出 DOS 控制台的方法是：win+R，然后输出 cmd，是不是看起来有点熟悉，没错，就是校验 JDK 安装是否成功是弹出的小黑屏。



DOS 命令有很多，我们不需要学习所有 DOS 命令，只需要学习我们 Java 中要用到的常用 DOS 命令，内容如下：

- 清屏：cls；
 - 当 DOS 控制台内容过多时，可以使用清屏命令，等同与“擦黑板”；
 - 例如：C:\>cls，然后按回车键。
- 切换盘符：:；
 - 默认情况下我们是在 C 盘中，如果想去其他盘符可以使用该命令；
 - 例如：C:\>E:，然后按回车键就到了 E 盘下。
- 进入指定目录：cd；
 - 当我们想进入某个目录时可以使用该命令；
 - 例如：D:\>cd develop，这样就进入了 D:\develop 目录，当然你需要保证这个目录是存在的；
 - 例如：D:\>cd develop\Java，这样就进入了 D:\develop\Java 目录。
- 返回上一级目录：cd..；0
 - 当你需要返回到上一级目录时可以使用该命令；
 - 例如：D:\develop\Java>cd..，然后按回车，就会返回到 D:\develop 目录下。
- 返回盘符根目录：cd\；
 - 当需要返回到当前盘符根目录下可以使用该命令；
 - 例如：D:\develop\Java>cd\，然后按回车，这样就返回到 D:/目录下了。
- 显示当前目录下的文件和子目录信息：dir；
 - 当需要查看当前目录下所有文件和子目录信息时可以使用该命令；
- 运行应用程序：程序完整名称 参数
 - 当需要运行某个应用程序时可以使用该命令；

- 例如要运行记事本程序时：D:\>C:\windows\notepad.exe，这样就打开了记事本程序，这与你到 C:\windows 目录下双击该程序图标的效果相同，因为该程序支持双击运行；
- 还可以省略 “.exe” 后缀，例如：D:\>C:\windows\notepad，效果同上；

1.3 Java 开发环境概述

在开始学习 Java 编程之前，我们首先需要安装 Java 开发环境（JDK：Java 开发工具包 Java Development Kit）。

你可能会问：“我们为什么要安装 JDK 呢”，其实任何事物都需要有他的最基本的生存环境，就像人类也需要最基本的生存环境（天、地、水、空气.....）。开发 Java 程序也不能例外，它也需要最基本的“生存环境”，就是 Java 开发环境（JDK）。

至于 JDK 中都包含了什么东西，现在还不是讲解的时候。我们现在的任务就是马上把它下载、安装，并校验安装是否成功。

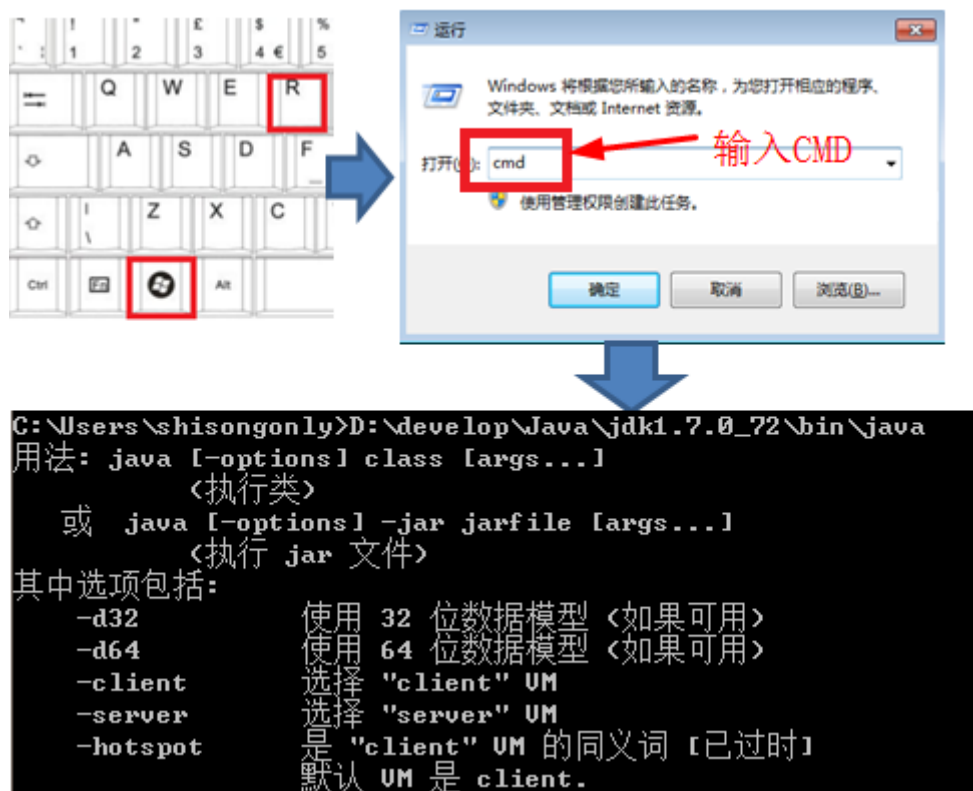
我们课程中使用的 JDK 版本是 JDK7，当前最新版本是 JDK8。

1.4 JDK 下载&安装&校验

JDK 的下载和安装请各位严格按照《JDK 下载安装文档 0.doc》中的要求操作，如果没有按照要求操作可能会引起意想不到的问题。这里就不再赘述！

下面我们来聊聊如何校验 JDK 安装是否成功的方法。你无需了解为什么要这么做，只需要严格按照下面的步骤操作即可：

1. 在键盘上按：WIN+R 键，这时会弹出运行窗口；
2. 在运行窗口中输入：CMD，按“确定”按钮，这时会弹出 DOS 窗口；
3. 在 DOS 窗口中输入：JDK 安装路径\bin\java.exe。



如果结果与图片中相同，恭喜你，安装成功；否则安装失败!!! 下面给出最常见的安装失败样例：

1. 系统找不到指定的路径：可能是因为 JDK 安装路径输入错误；
2. 不是内部或外部命令，也不是可运行的程序或批处理文件：安装失败。

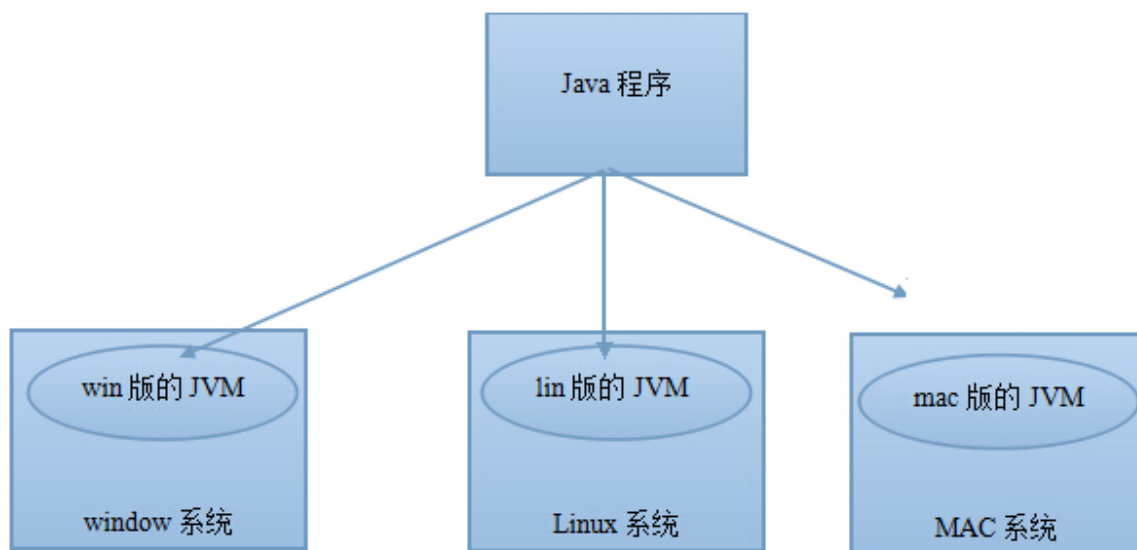
1.5 JDK 与 JRE

在安装的过程中各位可能注意到了 JRE 这个名词，下面我们来聊聊 JDK 和 JRE 的关系和区别。

- JDK：它是 Java 开发运行环境，在程序员的电脑上当然要安装 JDK；
- JRE：Java Runtime Environment 它是 Java 运行环境，如果你不需要开发只需要运行 Java 程序，那么你可以安装 JRE。例如程序员开发出的程序最终卖给了用户，用户不用开发，只需要运行程序，所以用户在电脑上安装 JRE 即可。
- JDK 包含了 JRE。

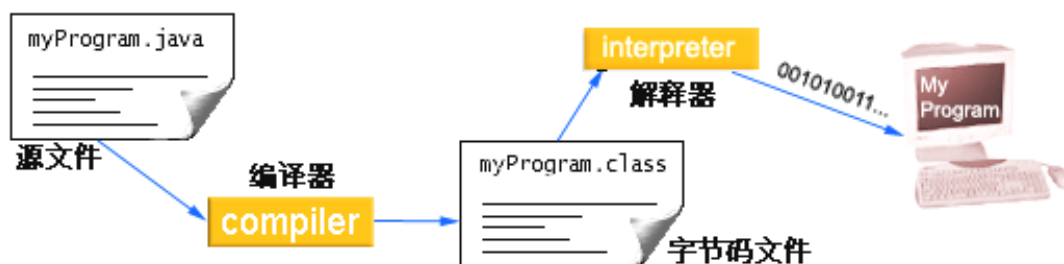
1.6 跨平台特性

- 平台指的是操作系统（Windows, Linux, Mac）。
- Java 程序可以在任意操作系统上运行，一次编写到处运行
- 实现跨平台需要依赖 Java 的虚拟机 JVM（Java Virtual Machine）



第2章 Hello World 案例

2.1 Java 程序介绍



在开始案例之前，我们需要了解一下 Java 应用程序的编写流程。

通过上图我们可以了解到编写的程序大致如下；

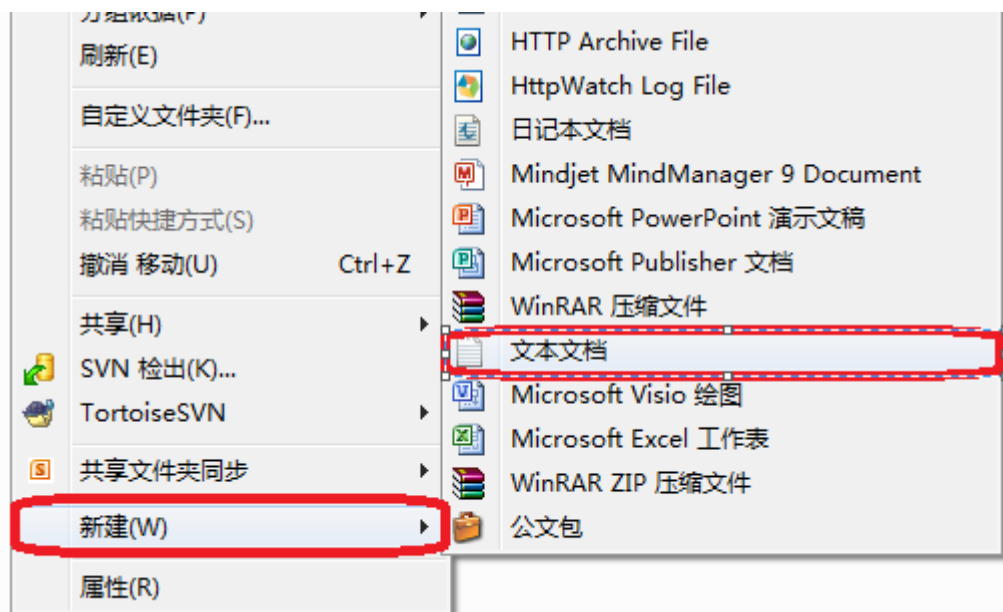
1. 源文件：编写 Java 源文件（我们也称之为源代码文件），它的扩展名为.java；
2. 编译：然后通过编译器把源文件编译成字节码文件，字节码文件扩展名为.class；
3. 运行：最后使用解释器来运行字节码文件。

编译和运行操作需要使用 DOS 命令，所以在编写源代码文件之后，我们要学习常用 DOS 命令，再去学习编译和运行。

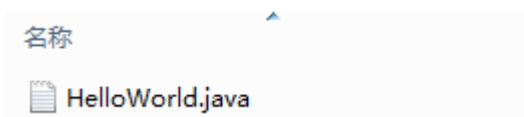
2.2 源代码编写

2.2.1 创建 HelloWorld.java 文件

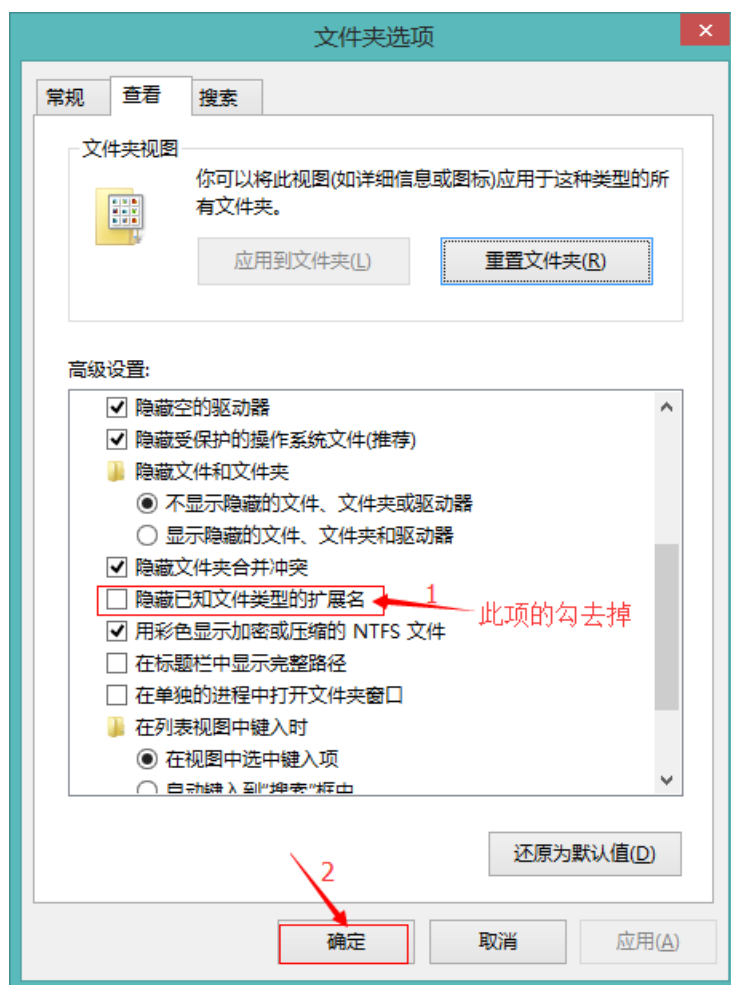
Java 源代码编写只需要使用最普通的文本编辑器编写即可，例如 Windows 系统自带的 notepad 即可。如果你不知道什么是 notepad，那么你可以在空白处点击鼠标右键→新建→文本文档来创建文本文件。



然后修改文件名称为 HelloWorld.java，注意文件名称的大小写。以及是否已经修改了文件的扩展名。



如果看不到文件后缀名，可通过窗口上方“查看”菜单中的“文件夹选择”找到“查看”选项卡，在“查看选择卡”的高级设置找到“隐藏已知文件类型的扩展名”选项，去掉前面的勾，点击“确定”即可显示文件后缀名了。如下图



2.2.2 编写源程序代码

下面是完整的内容：

HelloWorld.java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("HelloWorld");  
    }  
}
```

注意，大家在编写源代码时一定要注意大小写。总而言之，一模一样就对了。

下面我们对源代码进行解析，HelloWorld 案例的源代码一共分为三个部分，分别是：**类**、**主方法**，以及**输出语句**。

类是最外层的部分，其中 `public class` 是必须要写的命令，其中 HelloWorld 是类名称，要求必须与源文件名称相同。因为我们的源文件名称为 HelloWorld.java，所以类名称必须是 HelloWorld。类有一对大括号，类的内容存放在这对大括号中。你可能会问什么是类，我的答案是先不要去管什么是类，现在你需要记下来下面的代码，即定义类的语法。

```
public class HelloWorld {  
}
```

主方法是类的内容，所以主方法放到类的大括号中。现在不需要了解更多，只需要把主方法的内容背下来即可。主方法也有一对大括号，用来存放主方法的内容。

```
public static void main(String[] args) {  
}
```

输出语句是主方法的内容，所以输出语句需要放到主方法的大括号中。输出语句也需要背下来。输出语句最后有一对圆括号，在圆括号中有一句话用双引号括起来：**Hello World!**。在程序运行时输出它。

```
System.out.println("HelloWorld!");
```

到这里我们的源代码就已经编写完毕了，还要再次提醒大家编写源代码时要注意大小写。文件名称和文件内容都要注意大小写。

2.3 编译源代码文件

学习完 DOS 命令后我们就可以来学习编译源代码文件了。编译操作需要使用编译器来完成，在安装 JDK 时已经安装了编译器，它属于 JDK 的一部分。可以到 JDK 目录下的 bin 目录下找到它，它的名称为：javac.exe。



注意，javac.exe 并不支持双击运行，所以必须使用 DOS 命令来运行它。

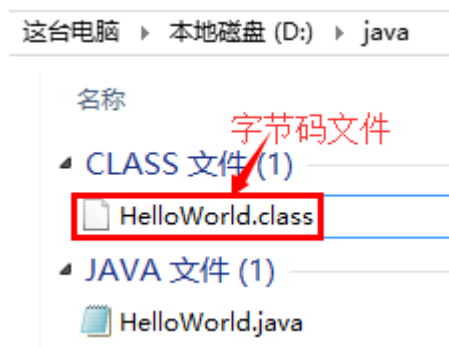
下面我们来介绍一下使用编译器来编译 Java 源文件的格式：

格式：编译器程序完整名称 源文件完整名称

让我们来分析一下：

- 我的 JDK 安装在 D:\develop\Java\jdk1.7.0_72 目录，那么编译器完整名称为：D:\develop\Java\jdk1.7.0_72\bin\javac.exe；
- 我的源文件存放在：D:\java\HelloWorld.java；
- 编译命令为：D:\develop\Java\jdk1.7.0_72\bin\javac.exe D:\java\HelloWorld.java。

编译之后会在源文件同目录下生成字节码文件，字节码文件的扩展名为.class。



请注意，在编译时控制台上不会有输出，如果有输出说明源代码有错误，所以大家一旦编译出错，就要去查看源文件中是否存在错误。

```
C:\Users\shisongonly>D:\develop\Java\jdk1.7.0_72\bin\javac d:\java\HelloWorld.java
C:\Users\shisongonly>
```

2.4 运行字节码文件

运行字节码文件需要使用 `java.exe` 命令，它与 `javac.exe` 在同一目录下。与 `javac.exe` 相同，`java.exe` 也不支持双击运行，所以也必须要使用 DOS 命令来运行。运行字节码文件与编译有点区别：

- 进行字节码文件所在目录：即进入 `D:\java` 目录；
- `D:\develop\Java\jdk1.7.0_72\bin\java.exe HelloWorld`；

注意，在运行 `HelloWorld.class` 文件时不能给出 “.class”，切记!!!

```
d:\java>D:\develop\Java\jdk1.7.0_72\bin\java HelloWorld
Hello World
d:\java>
```

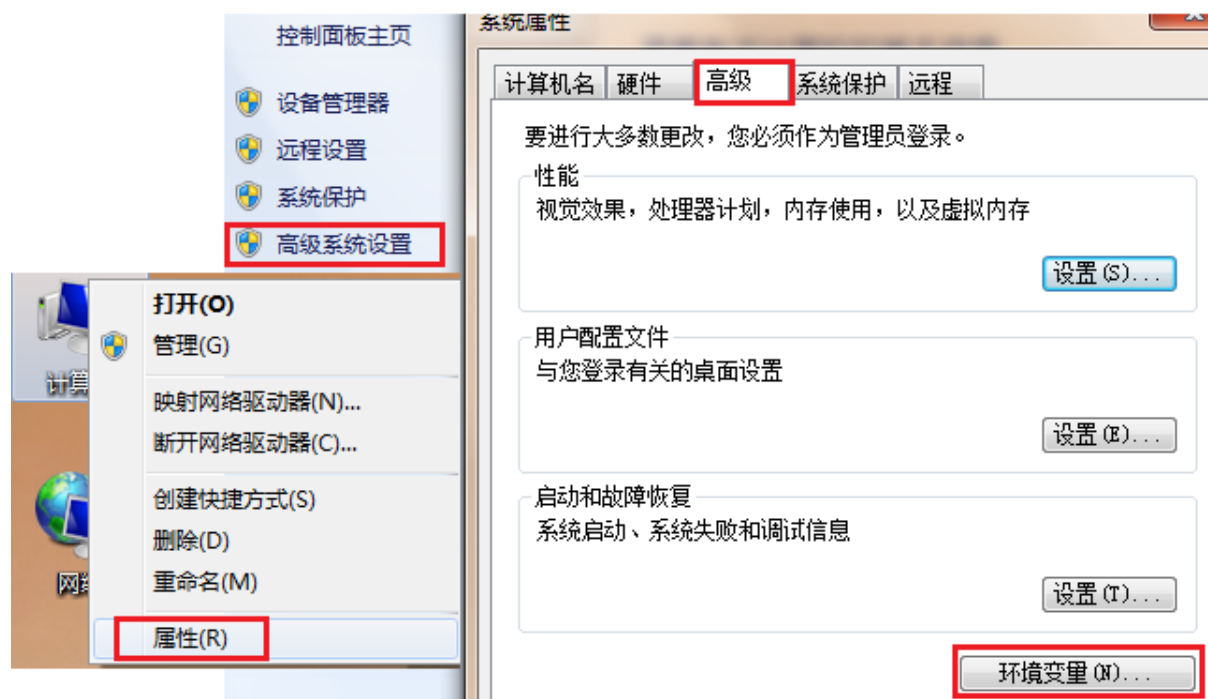
2.5 简化编译和运行操作

我们发现每次程序编译与运行都要输入 `D:\develop\Java\jdk1.7.0_72\bin` 后，才能使用 `javac.exe` 与 `java.exe`，这样的方式很麻烦，那么，能否不输入 `D:\develop\Java\jdk1.7.0_72\bin` 呢？是可以的，通过配置 Windows 的 PATH 环境变量来完成。

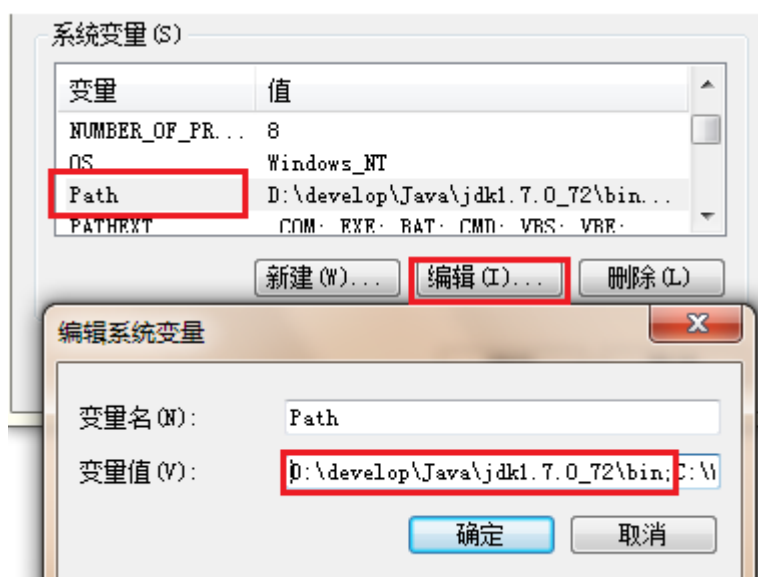
PATH 变量中保存着多个路径，当我们在 DOS 控制台中使用的程序只给出程序名称，而没有给出完整路径时，那么 Windows 系统会到 PATH 变量保存的路径中去查找程序。我们如果把 “`D:\develop\Java\jdk1.7.0_72\bin`” 路径保存到 PATH 变量中，那么在我们使用 `javac` 或 `java` 时，Windows 系统就会自动到 “`D:\develop\Java\jdk1.7.0_72\bin`” 路径下查找 `javac` 和 `java` 了。

- 配置步骤：

鼠标右键点击计算机 → 属性 → 高级系统设置 → 点击环境变量 → 找到系统变量中的 PATH → 点击编辑按钮。



将 JDK 安装目录\bin 路径（我的电脑上的 D:\develop\Java\jdk1.7.0_72\bin）配置到 PATH 变量中，用英文分号与其他变量隔开。



注意：配置 PATH 后文件的访问顺序：先访问当前路径，如果当前路径没有该文件，则再访问 PATH 配置的路径。

接下来我们测试一下：

- 把原有的 HelloWorld.class 删除；
- 进入 HelloWorld.java 所在目录：
 - 切换盘符到 E 盘：C:/>D:；
 - 进入 java 目录：D:/>cd java；
 - 编译：D:/java/>javac HelloWorld.java；
 - 运行：D:/java/>java HelloWorld；

```
C:\Users\Li>d:
D:\>cd java
D:\java>javac HelloWorld.java
D:\java>java HelloWorld
HelloWorld!
D:\java>
```

第3章 注释、关键字与标识符

3.1 程序注释

通常我们需要在源代码中添加文字用来对进行代码解释说明，但这些文字并不是 Java 代码的语法，会导致编译出错。这时我们可以使用注释来完成这一事项！

在编译时，编译器会忽略注释的存在，就好像注释内容不存在一样。所以注释并不会导致编译出错，同时注释还方便编写者和其他人阅读源代码，加强对代码的理解。

Java 中提供了三种注释方式，分别是：

- 单行注释 //注释内容
- 多行注释 /*注释内容*/
- 文档注释 /**注释内容*/

其中文档注释与多行注释作用基本相同，唯一的区别是文档注释可以使用 `javadoc.exe` 命令生成 API 文档。在这里我们不对其进行讲解，等大家对 API 有一定了解后在学习文档注释生成 API 文档的相关知识！

HelloWorld.java

```
/*
 * 实现步骤：
 * 1.定义一个类 class
 * 2.编写程序执行的入口方法，main 主方法
 * 3.通过输出语句 System.out.println() 将信息"HelloWorld!"打印在控制台上
 */
// 定义一个类 class
class HelloWorld {
    // 编写程序执行的入口方法，main 主方法
    public static void main(String[] args) {
        // 通过 Java 提供的输出语句，将信息"HelloWorld!"打印在控制台上
        System.out.println("HelloWorld!");
    }
}
```

建议大家今后在编写代码时先写思路、分析、步骤，然后再写代码。其中思路、分析、步骤都

使用注释放在源代码中。

3.2 关键字

关键字是被 Java 语言赋予特殊含义，具有专门用途的单词，比如之前接触的 `class`, `public`, `static`, `void` 均为 Java 已经预设好的。你可以先把 Java 关键字理解为“命令”！

Java 中关键字均为小写，下面是 Java 中所有关键字，你不需要去背，在今后的学习中重要的关键字会不断的出来。

<code>abstract</code>	<code>assert</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>class</code>	<code>const</code>
<code>continue</code>	<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>
<code>enum</code>	<code>extends</code>	<code>final</code>	<code>finally</code>	<code>float</code>
<code>for</code>	<code>goto</code>	<code>if</code>	<code>implements</code>	<code>import</code>
<code>instanceof</code>	<code>int</code>	<code>interface</code>	<code>long</code>	<code>native</code>
<code>new</code>	<code>package</code>	<code>private</code>	<code>protected</code>	<code>public</code>
<code>return</code>	<code>strictfp</code>	<code>short</code>	<code>static</code>	<code>super</code>
<code>switch</code>	<code>synchronized</code>	<code>this</code>	<code>throw</code>	<code>throws</code>
<code>transient</code>	<code>try</code>	<code>void</code>	<code>volatile</code>	<code>while</code>

你可以尝试看看 HelloWorld 案例中哪些是关键字，哪些不是！

3.3 标识符

学习完了关键字下面我们来学习一下什么是标识符。其实在 Java 程序中，除了关键字以外基本上都是标识符了。

其实标识符就是名称的意思，所有的名称都统称为标识符。Java 中经常要定义类、方法、变量（后面会学习到），在定义他们时总要给它们起名字，这些名字就是标识符。

这里我们要学习的是如何起名字，你可能会说起名字还要学吗？答案是当然要学了！我们要学习的就是标识符的规范。

- 组成元素
 - 英文字符：a-zA-Z
 - 数字：0-9
 - 符号：_与\$
- 标识符规则
 - 数字不能开头
 - 不可以使用关键字
 - 严格区分大小写，不限制长度起名时，尽量达到见名知意

第4章 数据

4.1 程序概述

软件是什么？其实软件就是用来处理数据的程序。可以把软件分为两个部分，一个部分是“数据”；另一个部分是“数据处理的逻辑”。

程序 = 数据 + 逻辑

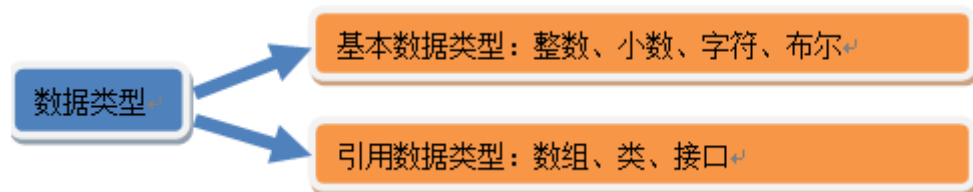
什么是数据？用电商来举例，用户就是数据，商品也是数据。那么什么是数据处理逻辑呢？购买商品的流程就是数据处理逻辑了。

程序开发离不开数据，其实说白了程序就是用来处理数据的。在 Java 中我们要学习数据的类型及数据量。

通常数据是会发生变化的，而数据的处理逻辑是不发生变化的。例如商品的数量种类是会变化的，用户的数量也是会变化的，但购买的流程通常不会变化。

4.2 数据类型

在生活中数据是有类型的，例如人的姓名是字符串类型，人的年龄是正整数类型。在 Java 中数据也是有类型的（任何数据必然会有类型），下面是 Java 中数据的类型：



基本数据类型是 Java 语言中内置的类型，分别是整数类型、小数类型、字符类型、布尔类型。这四类基本类型是最简单、最基础的类型。

引用数据类型是强大的数据类型，它是基于基本数据类型创建的。JavaSE 中提供了一个超级类库，类库中包含了近万种引用数据类型。不过现在我们先要学习的是基本类型！

4.3 常量

Java 中的数据量分为常量和变量，我们先来学习常量！

常量就是不变的数据量，例如 100 就是常量，任何数据量都有其类型，那么 100 这个常量是什么类型呢？回忆一下基本类型中有四类，分别是整数、小数（浮点数）、字符、布尔。那么 100 是哪一类型呢？没错，就是整数常量!!!

- 整数类型
 - 十进制表示方式：正常数字 如 13、25 等
 - 二进制表示方式：以 0b(0B)开头 如 0b1011 、0B1001
 - 十六进制表示方式：以 0x(0X)开头 数字以 0-9 及 A-F 组成 如 0x23A2、0xa、0x10
 - 八进制表示方式：以 0 开头 如 01、07、0721
- 小数类型
 - 如 1.0、-3.15、3.168 等

- 布尔类型
 - true、false
- 字符类型
 - 如 'a', 'A', '0', '家'
 - 字符必须使用 ' ' 包裹，并且其中只能且仅能包含一个字符。
- 字符串类型
 - 字符串 String 类型是一种引用类型，我们先了解作为常量类型的使用方式
 - 如 “我爱 Java”，“0123”，“”，“null”
 - 字符串必须使用 “ ” 包裹，其中可以包含 0~N 个字符。

回顾 HelloWorld 案例，其中 `System.out.println("Hello World!")` 的圆括号中放的就是一个字符串。那么我们是否可以把圆括号中的字符串换成其他字符串呢？答案是可以的！不只是可以换成其他字符串，还可以把圆括号中的字符串换成其他类型的常量，例如：

我们通过编写代码来实际感受一下：`Const.java`

```
/*
 * 常量的演示
 */
public class Const {
    public static void main(String[] args) {
        // 整数类型
        // 10 进制
        System.out.println(100);
        System.out.println(12345);
        System.out.println( 10 );
        // 2 进制
        System.out.println( 0b1010 );
        // 8 进制
        System.out.println( 012 );
        // 16 进制
        System.out.println(0xA);
        // 小数类型
        System.out.println(3.15);
        // 字符类型
        System.out.println('a');
        System.out.println('A');
        System.out.println('0');
        System.out.println('家');
        // 布尔类型
        System.out.println(true);
        System.out.println(false);
        // 字符串类型
        System.out.println("我爱 Java");
        System.out.println("0123");
    }
}
```

```
}
```

第5章 总结

5.1 Java 环境

- 下载和安装 JDK
- 配置环境变量
 - 计算机->属性->高级系统设置->高级->环境变量->系统变量 path->jdk 安装目录 bin

5.2 注释

- Java 的三种注释写法
- `//` 单行
- `/**` 多行
- `/**` 文档注释

5.3 标识符

- 定义
 - 被 Java 定义好的，赋予特殊含义单词
 - 全部采用小写字母
 - `class public static void true false`

5.4 命名规范

- 自定义的内容，类名，方法名，变量名
- 包含字母大小写，数字，下划线和美元符，不能数字开头
- 不允许是关键字

5.5 数据类型

- 基本数据类型
 - 整数 `0 1 2`
 - 浮点 `0.1 1.0 2.2`
 - 布尔 `true false`
 - 字符 `'a'`

- 字符串常量 "" 任意长度